



Facultad de Ingeniería y Administración

Clasificación Tabular e Imagen con Redes Neuronales en PyTorch

Juan David Castrillón Martínez

Proyecto Final

Inteligencia Artificial para Ingeniería Agroindustria

5 de mayo de 2026

Índice

1. Introducción 2

2. Componente Tabular 2

2.1. Descripción del dataset 2

2.2. Preprocesamiento 2

2.3. Modelo: Perceptrón Multicapa (MLP) 3

2.3.1. Fundamento matemático 3

2.3.2. Arquitectura 3

2.4. Entrenamiento 4

2.4.1. Manejo del desbalance de clases 4

2.4.2. Evolución de la pérdida 5

2.5. Resultados y evaluación 5

2.6. Análisis 5

3. Componente de Imágenes 5

3.1. Descripción del dataset 6

3.2. Preprocesamiento 6

3.3. Modelo: Red Neuronal Convolucional (CNN) 6

3.3.1. Fundamento matemático 6

3.3.2. Arquitectura y flujo de dimensiones 7

3.4. Entrenamiento 7

3.4.1. Evolución de la pérdida 8

3.5. Resultados y evaluación 8

3.6. Análisis 9

4. Conclusiones 9

1. Introducción

Este proyecto presenta el desarrollo de dos modelos de clasificación supervisada usando técnicas de inteligencia artificial implementadas en PyTorch. El trabajo abarca dos dominios distintos del problema agroindustrial:

- **Componente tabular:** clasificación binaria sobre un dataset de variables numéricas, usando un Perceptrón Multicapa (MLP).
- **Componente de imágenes:** clasificación multiclase de hojas de papa a partir de fotografías, usando una Red Neuronal Convolucional (CNN).

En ambos casos se siguió un flujo completo: exploración del dataset, preprocesamiento, diseño del modelo, entrenamiento, evaluación con métricas y análisis crítico de los resultados.

2. Componente Tabular

2.1. Descripción del dataset

El dataset utilizado es `phpnThNfi.arff`, cargado desde el repositorio del curso. Contiene **4839 observaciones** con **5 variables de entrada numéricas** (V1 a V5) y una **variable objetivo binaria** denominada *Class*, con valores '1' (clase 0) y '2' (clase 1).

El dataset presenta un **desbalance de clases significativo**:

Clase	Muestras (train)	Proporción
0 (original '1')	3,662	94.6 %
1 (original '2')	209	5.4 %
Total train	3,871	
Total test	968	

Cuadro 1: Distribución de clases en el dataset tabular.

2.2. Preprocesamiento

El pipeline implementado en `preprocess.py` realiza los siguientes pasos:

1. Carga del archivo ARFF con `scipy.io.arff` y conversión a DataFrame.
2. Limpieza de la variable objetivo: decodificación de `bytes` a `string`.
3. Separación de $X = [V1, V2, V3, V4, V5]$ e y .
4. Codificación con `LabelEncoder`: '1' $\rightarrow$ 0, '2' $\rightarrow$ 1.
5. División 80 %/20 % con muestreo estratificado.
6. Escalado con `StandardScaler` ajustado solo sobre train: $x_{\text{norm}} = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$

7. Conversión a tensores PyTorch.

Listing 1: Pipeline de preprocesamiento (preprocess.py)

```

1 def preprocess_pipeline(filepath, feature_cols=None,
2                           target_col="Class", test_size=0.2,
3                           random_state=42):
4     if feature_cols is None:
5         feature_cols = ["V1", "V2", "V3", "V4", "V5"]
6
7     df = load_arff(filepath)
8     df = clean_target(df, target_col)
9     X, y = split_features_target(df, feature_cols, target_col)
10    y, le = encode_target(y)
11    X_train, X_test, y_train, y_test = split_data(
12        X, y, test_size, random_state)
13    X_train, X_test, scaler = scale_features(X_train, X_test)
14    X_train_t, X_test_t, y_train_t, y_test_t = to_tensors(
15        X_train, X_test, y_train, y_test)
16    return {"X_train": X_train_t, "X_test": X_test_t,
17           "y_train": y_train_t, "y_test": y_test_t,
18           "num_classes": len(le.classes_)}

```

2.3. Modelo: Perceptrón Multicapa (MLP)

2.3.1. Fundamento matemático

Un MLP transforma la entrada $\mathbf{x} \in \mathbb{R}^5$ mediante capas sucesivas de transformaciones lineales y no lineales. Cada capa realiza:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad \mathbf{a}^{(l)} = \text{ReLU}(\mathbf{z}^{(l)}) = \max(0, \mathbf{z}^{(l)})$$

La capa de salida produce logits $\mathbf{z}^{(\text{out})} \in \mathbb{R}^2$ sin activación, ya que `CrossEntropyLoss` incorpora `LogSoftmax` internamente.

2.3.2. Arquitectura

Capa	Tipo	Entrada	Salida
1	Linear + ReLU	5	16
2	Linear + ReLU	16	8
3	Linear (salida)	8	2

Cuadro 2: Arquitectura del MLP.

Listing 2: Definición del MLP (model.py)

```

1 class MLP(nn.Module):
2     def __init__(self, input_dim: int, num_classes: int):
3         super().__init__()

```

```

4         self.fc1    = nn.Linear(input_dim, 16)
5         self.relu1  = nn.ReLU()
6         self.fc2    = nn.Linear(16, 8)
7         self.relu2  = nn.ReLU()
8         self.fc3    = nn.Linear(8, num_classes)
9
10        def forward(self, x):
11            x = self.relu1(self.fc1(x))
12            x = self.relu2(self.fc2(x))
13            return self.fc3(x)    # logits crudos, sin Softmax

```

2.4. Entrenamiento

2.4.1. Manejo del desbalance de clases

Dado el desbalance severo (ratio $\approx 17:1$), se aplicaron pesos de clase con la fórmula estándar equivalente a `class_weight='balanced'` de scikit-learn:

$$w_i = \frac{n_{\text{total}}}{n_{\text{clases}} \times n_i}$$

Los pesos resultantes con el dataset fueron:

- Clase 0 (3.662 muestras): $w_0 = 0,5285$
- Clase 1 (209 muestras): $w_1 = 9,2608$

Listing 3: Cálculo de pesos de clase (`train.py`)

```

1  def compute_class_weights(y_train):
2      class_counts = torch.bincount(y_train)
3      n_total      = len(y_train)
4      n_classes    = len(class_counts)
5      weights = n_total / (n_classes * class_counts.float())
6      return weights
7
8  weights      = compute_class_weights(y_train)
9  criterion    = nn.CrossEntropyLoss(weight=weights)
10 optimizer    = torch.optim.Adam(model.parameters(), lr=0.001)

```

2.4.2. Evolución de la pérdida

Época	Loss	Época	Loss
1	0.6943	11	0.6834
2	0.6933	12	0.6823
3	0.6922	13	0.6812
4	0.6911	14	0.6801
5	0.6900	15	0.6790
6	0.6889	16	0.6779
7	0.6878	17	0.6768
8	0.6867	18	0.6757
9	0.6856	19	0.6746
10	0.6845	20	0.6735

Cuadro 3: Evolución de la pérdida por época — componente tabular.

2.5. Resultados y evaluación

Accuracy en test: 67.98 %

Matriz de confusión:

$$CM = \begin{bmatrix} 622 & 294 \\ 16 & 36 \end{bmatrix}$$

Clase	Aciertos	Total	% por clase
0	622	916	67.9 %
1	36	52	69.2 %

Cuadro 4: Detalle de aciertos por clase — componente tabular.

2.6. Análisis

El modelo logra aprender **ambas clases de forma equilibrada** gracias a los pesos aplicados, con tasas de acierto similares (67.9 % y 69.2 %). Esto contrasta con el comportamiento sin pesos, donde el modelo predecía siempre la clase mayoritaria con accuracy ficticia del 94.6 % y 0 % de aciertos en la clase minoritaria.

La descenso gradual y constante de la pérdida ($0.6943 \rightarrow 0.6735$) indica aprendizaje sostenido, aunque la convergencia lenta sugiere que el dataset es intrínsecamente difícil de separar con estas features. Arquitecturas más profundas o técnicas de feature engineering podrían mejorar el resultado.

3. Componente de Imágenes

3.1. Descripción del dataset

El dataset `potato_subset` contiene **456 imágenes** de hojas de papa clasificadas en tres categorías:

- `Potato___Early_blight` — tizón temprano
- `Potato___Late_blight` — tizón tardío
- `Potato___healthy` — hoja sana

Conjunto	Imágenes	Proporción
Entrenamiento	364	80 %
Prueba	92	20 %
Total	456	100 %

Cuadro 5: División del dataset de imágenes.

3.2. Preprocesamiento

Las imágenes se cargan con `torchvision.datasets.ImageFolder`, que detecta las clases automáticamente desde la estructura de carpetas. El pipeline de transformaciones es:

1. **Resize** a 128×128 px para unificar dimensiones.
2. **ToTensor**: $(H \times W \times C)$ uint8 $[0, 255] \rightarrow (C \times H \times W)$ float32 $[0, 1]$.
3. **Normalize** canal a canal: $x_{\text{norm}} = (x - 0,5)/0,5 \in [-1, 1]$

Listing 4: Transformaciones e ImageFolder (`dataset.py`)

```

1 transform = transforms.Compose([
2     transforms.Resize((128, 128)),
3     transforms.ToTensor(),
4     transforms.Normalize(mean=[0.5, 0.5, 0.5],
5                             std=[0.5, 0.5, 0.5]),
6 ])
7 dataset = datasets.ImageFolder(root=data_path, transform=transform)
8 train_loader = DataLoader(train_dataset, batch_size=32, shuffle=
    True)
9 test_loader = DataLoader(test_dataset, batch_size=32, shuffle=
    False)

```

3.3. Modelo: Red Neuronal Convolutacional (CNN)

3.3.1. Fundamento matemático

Una CNN extrae características espaciales mediante filtros aprendibles. La operación de convolución 2D es:

$$S[i, j] = \sum_m \sum_n I[i + m, j + n] \cdot K[m, n]$$

donde I es el mapa de entrada y K el filtro (kernel). El **MaxPooling** reduce la resolución tomando el máximo de cada ventana 2×2 , logrando invariancia a traslaciones y reduciendo parámetros.

3.3.2. Arquitectura y flujo de dimensiones

Capa	Tipo	Entrada	Salida
Conv1 + ReLU	Conv2d(3, 16, k=3, p=1)	($B, 3, 128, 128$)	($B, 16, 128, 128$)
MaxPool1	MaxPool2d(2, 2)	($B, 16, 128, 128$)	($B, 16, 64, 64$)
Conv2 + ReLU	Conv2d(16, 32, k=3, p=1)	($B, 16, 64, 64$)	($B, 32, 64, 64$)
MaxPool2	MaxPool2d(2, 2)	($B, 32, 64, 64$)	($B, 32, 32, 32$)
Flatten	—	($B, 32, 32, 32$)	($B, 32768$)
FC1 + ReLU	Linear(32768, 64)	($B, 32768$)	($B, 64$)
FC2 (salida)	Linear(64, 3)	($B, 64$)	($B, 3$)

Cuadro 6: Arquitectura CNN y flujo de dimensiones (B = tamaño de lote).

Listing 5: Definición de la CNN (model.py)

```

1 class CNN(nn.Module):
2     def __init__(self, num_classes: int):
3         super().__init__()
4         self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
5         self.relu1 = nn.ReLU()
6         self.pool1 = nn.MaxPool2d(2, 2)
7         self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
8         self.relu2 = nn.ReLU()
9         self.pool2 = nn.MaxPool2d(2, 2)
10        self.flatten = nn.Flatten()
11        self.fc1 = nn.Linear(32 * 32 * 32, 64)
12        self.relu3 = nn.ReLU()
13        self.fc2 = nn.Linear(64, num_classes)
14
15    def forward(self, x):
16        x = self.pool1(self.relu1(self.conv1(x)))
17        x = self.pool2(self.relu2(self.conv2(x)))
18        x = self.flatten(x)
19        x = self.relu3(self.fc1(x))
20        return self.fc2(x) # logits crudos, sin Softmax

```

3.4. Entrenamiento

El modelo se entrenó durante **10 épocas** iterando sobre batches de 32 imágenes. Se usó **CrossEntropyLoss** sin pesos (clases balanceadas) y **Adam** con $\alpha = 0,001$.

Listing 6: Bucle de entrenamiento por batches (`train.py`)

```

1 for epoch in range(1, epochs + 1):
2     running_loss = 0.0
3     for images, labels in train_loader:
4         images, labels = images.to(device), labels.to(device)
5         logits = model(images)
6         loss = criterion(logits, labels)
7         optimizer.zero_grad()
8         loss.backward()
9         optimizer.step()
10        running_loss += loss.item()
11    avg_loss = running_loss / len(train_loader)
12    print(f"    Epoca {epoch}/{epochs} | Loss: {avg_loss:.4f}")

```

3.4.1. Evolución de la pérdida

Época	Loss promedio
1	0.7736
2	0.3956
3	0.4436
4	0.2250
5	0.1902
6	0.1388
7	0.1166
8	0.0755
9	0.0325
10	0.0226

Cuadro 7: Evolución de la pérdida por época — componente de imágenes.

3.5. Resultados y evaluación

Accuracy en test: 92.39 %

Métrica	Valor
Accuracy en test	92.39 %
Loss inicial (época 1)	0.7736
Loss final (época 10)	0.0226
Reducción de pérdida	97.1 %

Cuadro 8: Resumen de resultados — componente de imágenes.

3.6. Análisis

La caída de la pérdida de 0.7736 a 0.0226 en solo 10 épocas indica que la CNN aprende características discriminativas de forma efectiva. La accuracy del **92.39 %** es un resultado sólido considerando el tamaño reducido del dataset (456 imágenes).

El leve repunte en época 3 (0.4436 tras 0.3956) es normal en entrenamiento por batches con `shuffle=True`. A partir de época 4 la tendencia es consistentemente decreciente.

La pérdida muy baja al final (0.0226) sugiere posible **sobreajuste**. Para producción se recomendaría aplicar *data augmentation* (rotaciones, volteos, cambios de brillo) y *Dropout* como regularización.

4. Conclusiones

Este proyecto demostró el desarrollo completo de dos pipelines de clasificación supervisada en PyTorch aplicados a problemas agroindustriales reales.

Componente tabular:

- El desbalance severo (ratio 17:1) no puede ignorarse: sin manejo, el modelo produce accuracy ficticia del 94.6 % con 0 % de aciertos en la clase minoritaria.
- Los pesos de clase con $w_i = n_{\text{total}} / (n_{\text{clases}} \times n_i)$ logran tasas equilibradas de 67.9 % y 69.2 % por clase.
- La convergencia lenta sugiere que el problema requiere más datos o features más discriminativas.

Componente de imágenes:

- La CNN alcanza 92.39 % de accuracy en 10 épocas, demostrando la efectividad de las arquitecturas convolucionales para clasificación de imágenes agrícolas.
- La pérdida baja final indica posible sobreajuste; en producción se recomendaría *data augmentation* y regularización.

Reflexión: La elección correcta de la función de pérdida, el manejo del desbalance y el preprocesamiento adecuado son tan importantes como la arquitectura. La comprensión del problema precede a la solución técnica.